INSTITUTO FEDERAL DO ESPÍRITO SANTO
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS

GESTÃO OPERACIONAL E CONTROLE DE LANCHONETES: DESENVOLVIMENTO DE UM SISTEMA INFORMATIZADO PARA AUTOMAÇÃO DE PROCESSOS

ALEGRE
2026

PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS

GESTÃO OPERACIONAL E CONTROLE DE LANCHONETES: DESENVOLVIMENTO DE UM SISTEMA INFORMATIZADO PARA AUTOMAÇÃO DE PROCESSOS

Trabalho de Conclusão de Curso apresentado à Coordenadoria do Curso de Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia do Espírito Santo, como requisito parcial para a obtenção do título de Analista em Desenvolvimento de Sistemas.

Orientador: Prof. D.Sc Carlos Alexandre Siqueira da Silva

ALEGRE

2026

Dados Internacionais de Catalogação-na-Publicação (CIP)
(Biblioteca Nilo Peçanha do Instituto Federal do Espírito Santo)

X999y

Gestão Operacional e Controle de Lanchonetes: Desenvolvimento de um Sistema Informatizado para Automação de Processos/ PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS. – 2026.

32 f: il; 30 cm.

Orientador: Carlos Alexandre Siqueira da Silva

Trabalho de Graduação – Instituto Federal do Espírito Santo, Curso de Análise e Desenvolvimento de Sistemas, 2026.

1. Redes neurais (Computação). 2. Expressão facial – Processamento de dados. 3. Sistemas de reconhecimento de padrões. 4. Processamento de imagens – Técnicas digitais. 5. Percepção de padrões. 6. Engenharia Elétrica. I. XXXXXXXX, XXXXXXXXXX XXXXXXXXXX. II. Instituto Federal do Espírito Santo. III. Título.

CDD 000.00

Elaborada por XXXXXXXXXXXXXXXXXXXXX – CRB-X/ES - XXX

PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS

GESTÃO OPERACIONAL E CONTROLE DE LANCHONETES: DESENVOLVIMENTO DE UM SISTEMA INFORMATIZADO PARA AUTOMAÇÃO DE PROCESSOS

Trabalho de Conclusão de Curso apresentado à Coordenadoria do Curso de Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia do Espírito Santo, como requisito parcial para a obtenção do título de Analista em Desenvolvimento de Sistemas.

Aprovado em 01 de Agosto de 2026

COMISSÃO EXAMINADORA

Prof. D.Sc Carlos Alexandre Siqueira da Silva
Instituto Federal do Espírito Santo
Orientador

Profa. Dra. Fulana de Tal
Instituto Federal do Espírito Santo
Examinadora

Prof. Dr. Cicrano de Tal
Instituto Federal do Espírito Santo
Examinador

A dedicatória é um elemento opcional. Contém o oferecimento do trabalho adeterminada pessoa ou a pessoas (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2011b).

DECLARAÇÃO DO AUTOR

Eu, **PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS**, declaro que este trabalho foi desenvolvido com base em meus próprios estudos e que não houve uso de material de terceiros, exceto o que foi citado.

Declaro para fins de pesquisa acadêmica, didática e técnico-científica, que este Trabalho de Conclusão de Curso pode ser parcialmente utilizado, desde que seja feita a devida referência à fonte e ao autor.

Alegre, 22 de abril de 2026

PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS

AGRADECIMENTOS

test elemento opcional. Localiza-se após a folha de aprovação e deve ser dirigido àqueles que realmente contribuíram, de maneira relevante,para a elaboração do trabalho. Deve-se utilizar uma linguagem simples (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2011b).

Epigrafe: É um elemento opcional. É uma citação relacionada ao assunto do trabalho desenvolvido, seguida da indicação de autoria (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2011b). Deve-se seguir as regras do uso da citação NBR 10.520/2002.

RESUMO

Este trabalho apresenta o desenvolvimento de uma aplicação web destinada ao apoio da gestão operacional de uma lanchonete. A proposta parte da identificação de dificuldades associadas ao uso de registros manuais, como erros no lançamento de pedidos, demora no atendimento, inconsistências no controle de produtos e limitação do acesso a informações gerenciais. O sistema foi estruturado com o framework Laravel e a linguagem PHP, utilizando MySQL para a persistência dos dados e Redis como mecanismo de cache. A arquitetura adota o padrão Model–View–Controller e o controle de acesso baseado em funções, com o objetivo de separar responsabilidades e restringir as funcionalidades conforme o perfil do usuário. Entre as funções previstas estão o cadastro e a autenticação de usuários, o gerenciamento de produtos, a apresentação do catálogo, o carrinho de compras e o acompanhamento dos pedidos. O uso de Docker contribui para a padronização do ambiente de execução, enquanto Git e GitHub apoiam o controle de versões. Espera-se que a solução reduza a ocorrência de falhas operacionais, centralize as informações do estabelecimento e proporcione maior agilidade ao atendimento e ao acompanhamento das atividades.

Palavras-chave: Gestão operacional. Sistemas web. Lanchonetes. Laravel. Automação de processos.

ABSTRACT

This work presents the development of a web application designed to support the operational management of a snack bar. The proposal is based on difficulties associated with manual records, including order-entry errors, service delays, inconsistencies in product control, and limited access to management information. The system was structured with the Laravel framework and PHP, using MySQL for data persistence and Redis as a caching mechanism. Its architecture follows the Model–View–Controller pattern and role-based access control to separate responsibilities and restrict features according to each user profile. The planned features include user registration and authentication, product management, catalog display, a shopping cart, and order tracking. Docker contributes to a standardized execution environment, while Git and GitHub support version control. The solution is expected to reduce operational failures, centralize business information, and improve the speed of customer service and activity monitoring.

Keywords: Operational management. Web systems. Snack bars. Laravel. Process automation.

LISTA DE ILUSTRAÇÕES

Figura 1 – Fluxo de consulta ao Redis no sistema Flashfood 24
Figura 2 – Modelo físico do banco de dados do sistema Flashfood 30

LISTA DE TABELAS

Tabela 1 – Requisitos funcionais do sistema 27
Tabela 2 – Requisitos não funcionais do sistema 28

LISTA DE QUADROS

LISTA DE SIGLAS

Ifes	Instituto Federal do Espírito Santo
MVC	Model–View–Controller
PHP	PHP: Hypertext Preprocessor
RBAC	Role-Based Access Control (Controle de Acesso Baseado em Funções)
RNF	Requisitos Não Funcionais
RF	Requisitos Funcionais
SQL	Structured Query Language (Linguagem de Consulta Estruturada)
SGBD	Sistema de Gerenciamento de Banco de Dados

SUMÁRIO

1	**INTRODUÇÃO**	**14**
2	**PROBLEMA DE PESQUISA**	**15**
3	**JUSTIFICATIVA**	**16**
4	**OBJETIVOS**	**18**
4.1	OBJETIVO GERAL	18
4.2	OBJETIVOS ESPECÍFICOS	18
5	**REFERENCIAL TEÓRICO**	**19**
5.1	MYSQL	19
5.2	PHP	19
5.3	LARAVEL	20
5.4	GIT	20
5.5	GITHUB	20
5.6	BLADE (SISTEMA DE TEMPLATES)	21
5.7	VISUAL STUDIO CODE	21
5.8	CONTROLE DE ACESSO BASEADO EM FUNÇÕES (RBAC)	21
5.9	DOCKER	22
5.10	REDIS	23
5.10.1	**Cache em Aplicações Web**	**23**
5.10.2	**Redis como Camada de Cache**	**23**
5.10.3	**Escalabilidade Horizontal**	**25**
5.10.4	**Aplicação do Redis no Sistema Desenvolvido**	**25**
6	**REQUISITOS DE SOFTWARE**	**26**
6.1	REQUISITOS FUNCIONAIS	26
6.2	REQUISITOS NÃO FUNCIONAIS	26
7	**MODELAGEM DE SOFTWARE**	**29**
8	**CONCLUSÃO**	**31**
	REFERÊNCIAS	**32**

1 INTRODUÇÃO

Devido à convivência com o ambiente de trabalho de uma lanchonete, foram identificadas diversas necessidades passíveis de automação, uma vez que o estabelecimento não opera com sua eficiência máxima. Isso ocorre principalmente pela ausência de um sistema informatizado para controle de estoque, gerenciamento de vendas, controle de pedidos e emissão de comandas, entre outras funcionalidades essenciais.

A inexistência dessa ferramenta tem ocasionado erros operacionais, atrasos no atendimento e sobrecarga de trabalho para os funcionários. Esses fatores podem resultar na insatisfação dos clientes, perda de vendas e na criação de um ambiente de trabalho estressante e ineficiente, afetando negativamente proprietários, colaboradores e consumidores.

Diante desse cenário, surgiu a necessidade de desenvolver um sistema capaz de otimizar os processos internos e melhorar a eficiência operacional, sem a necessidade de aumento no quadro de funcionários. Além disso, busca-se elevar o volume de vendas por meio da melhoria na organização e execução das atividades da lanchonete.

Com o desenvolvimento do sistema, espera-se promover uma melhoria significativa no funcionamento do estabelecimento, tornando o ambiente mais organizado, eficiente e agradável. Ademais, objetiva-se aumentar a satisfação de clientes, funcionários e gestores, contribuindo para a fidelização dos clientes, incremento nas vendas e melhoria na qualidade dos serviços prestados, constituindo assim um diferencial competitivo no mercado.

2 PROBLEMA DE PESQUISA

O setor de alimentação fora do lar exige rapidez, controle e organização. Em uma lanchonete, o atendimento envolve atividades como recebimento de pedidos, preparação, controle de itens consumidos, atualização do estoque e pagamento. Quando essas etapas não são apoiadas por um sistema integrado, aumentam as possibilidades de erros humanos, informações desencontradas, atrasos no atendimento e dificuldades no controle das operações.

No estabelecimento tomado como referência para este trabalho, identificou-se a necessidade de organizar processos relacionados a produtos, pedidos, comandas, usuários e relatórios. A execução manual dessas atividades pode ocasionar erros de anotação, dificuldade para localizar pedidos, demora no fechamento de contas e baixa visibilidade sobre os produtos com maior saída. Além disso, a ausência de um controle formal de acesso dificulta a separação de responsabilidades entre os diferentes usuários do sistema. A falta de um banco de dados centralizado também pode comprometer a confiabilidade das informações e o acompanhamento do estoque e das vendas.

Diante desse contexto, torna-se relevante investigar como uma aplicação web pode contribuir para a automação das rotinas internas de uma lanchonete. Para isso, este trabalho propõe o desenvolvimento de um sistema utilizando Laravel e MySQL, apoiado por tecnologias como PHP, Docker, MVC, RBAC e Git/GitHub, buscando oferecer uma solução adequada ao contexto de pequenos estabelecimentos.

Diante disso, este trabalho busca responder à seguinte questão de pesquisa:

> **Como o desenvolvimento de um sistema informatizado utilizando Laravel e MySQL pode contribuir para a automação de processos e para a melhoria da gestão operacional de uma lanchonete?**

3 JUSTIFICATIVA

A informatização de processos operacionais tem se tornado um fator relevante para a competitividade de pequenos negócios. Em lanchonetes, a agilidade no atendimento e a precisão das informações influenciam diretamente a satisfação dos clientes, o desempenho da equipe e a capacidade de controle do gestor. Quando pedidos, produtos e estoque são administrados sem integração, o estabelecimento fica mais exposto a perdas, atrasos, retrabalho e decisões baseadas em percepções incompletas.

Este trabalho se justifica, inicialmente, pela necessidade prática de desenvolver uma solução capaz de apoiar a rotina de uma lanchonete real ou representativa desse tipo de negócio. A proposta busca reduzir a dependência de registros manuais e centralizar informações importantes em um sistema web, permitindo que diferentes perfis de usuário acessem funcionalidades compatíveis com suas responsabilidades. Dessa forma, pretende-se tornar o atendimento mais organizado e oferecer ao gestor melhores condições para acompanhar a operação.

Do ponto de vista acadêmico, o projeto favorece a aplicação integrada de conhecimentos de análise e desenvolvimento de sistemas, banco de dados, engenharia de software, segurança da informação, arquitetura de software e documentação técnica. A construção de uma aplicação com Laravel e MySQL permite explorar conceitos como MVC, ORM, migrações, autenticação, autorização, regras de negócio e testes funcionais. Além disso, o uso de Docker e Git/GitHub aproxima o desenvolvimento das práticas adotadas em ambientes profissionais.

Sob a perspectiva tecnológica, a escolha do Laravel se justifica pela maturidade do ecossistema PHP e pela existência de recursos que reduzem a complexidade de tarefas recorrentes em aplicações web. O MySQL é adequado por ser amplamente utilizado, possuir documentação consolidada e atender às necessidades de armazenamento relacional do sistema. O Docker contribui para evitar diferenças entre ambientes de desenvolvimento, enquanto o Git permite registrar a evolução do código-fonte e apoiar a rastreabilidade das alterações.

Também há uma justificativa social e econômica. Pequenos empreendedores nem

sempre dispõem de recursos para adquirir sistemas comerciais robustos ou excessivamente complexos para sua realidade. Uma solução acadêmica, construída a partir de requisitos bem definidos, pode demonstrar caminhos viáveis para modernização de rotinas e melhoria da organização interna. Ainda que o sistema proposto não substitua integralmente plataformas comerciais completas, ele pode atender a necessidades centrais e servir como base para futuras evoluções.

Portanto, a relevância deste trabalho está em articular uma demanda operacional concreta com uma proposta técnica fundamentada. O estudo busca produzir um artefato de software útil, documentado e avaliável, contribuindo tanto para a formação acadêmica do autor quanto para a discussão sobre automação de processos em pequenos estabelecimentos alimentícios.

4 OBJETIVOS

4.1 OBJETIVO GERAL

Desenvolver uma aplicação web com o framework Laravel para apoiar a gestão operacional de uma lanchonete, contemplando o cadastro e a autenticação de usuários, o gerenciamento de produtos, a exibição do catálogo para clientes e o processamento de pedidos, com persistência dos dados em MySQL.

4.2 OBJETIVOS ESPECÍFICOS

- Levantar os requisitos funcionais e não funcionais do sistema de gestão de lanchonete.
- Modelar o banco de dados MySQL para armazenar informações de usuários, produtos e pedidos.
- Desenvolver um mecanismo de autenticação e controle de acesso para funcionários e clientes.
- Implementar o cadastro, edição, listagem e exclusão de produtos.
- Desenvolver a funcionalidade de exibição de produtos na área do cliente.
- Implementar o módulo de carrinho de compras e processamento de pedidos.
- Gerenciar pedidos e permitir a atualização de seus estados para pendente, em preparo ou finalizado.
- Integrar a aplicação ao banco de dados MySQL, garantindo a persistência das informações.
- Testar o sistema para garantir sua usabilidade, desempenho e funcionamento correto.

5 REFERENCIAL TEÓRICO

Nesta seção, são apresentados os principais conceitos e fundamentos teóricos utilizados no desenvolvimento deste trabalho, abordando os conhecimentos necessários para a compreensão das tecnologias e metodologias empregadas no sistema.

5.1 MYSQL

De acordo com Milani (2007), o cenário de globalização e o avanço da automação, impulsionados pela popularização da internet, tornaram o armazenamento de dados um passo primordial para a migração de negócios para o ambiente digital. Nesse contexto, o MySQL surge para suprir essa demanda de organização de informações.

O MySQL é um Sistema Gerenciador de Banco de Dados (SGBD) relacional distribuído sob um modelo de licenciamento duplo, que inclui uma versão de código aberto. Embora tenha sido projetado inicialmente para aplicações de pequeno e médio porte, a ferramenta evoluiu para atender também a projetos de maior escala. Segundo Milani (2007), o MySQL apresenta robustez e recursos comparáveis aos de sistemas proprietários, como SQL Server e Oracle.

5.2 PHP

Conforme afirma Dall'Oglio (2015), o PHP — que originalmente significava Personal Home Page — é uma linguagem de programação criada por Rasmus Lerdorf no outono de 1994. Inicialmente, a ferramenta consistia em um conjunto de scripts escritos em linguagem C, destinados à criação de páginas dinâmicas para que o autor monitorasse o acesso ao seu currículo na internet. Com o aumento da popularidade entre outros usuários, Lerdorf adicionou novos recursos, como a interação com bancos de dados. Em 1995, o código-fonte foi liberado, permitindo a colaboração de novos desenvolvedores. Vale ressaltar que, durante um curto período, a linguagem também foi denominada FI (Forms Interpreter).

5.3 LARAVEL

De acordo com N.Ricardo (2025), o Laravel é um *framework* para o desenvolvimento de aplicações web em PHP. A ferramenta oferece recursos para tarefas recorrentes, como roteamento, acesso a bancos de dados, autenticação e organização da aplicação, podendo ser utilizada tanto no desenvolvimento do *back-end* quanto em aplicações completas.

5.4 GIT

Segundo Ferreira (2014), o Git é um sistema de controle de versão que permite registrar e acompanhar as alterações realizadas no código-fonte ao longo do desenvolvimento.

Seu fundador, Linus Torvalds, criador do sistema operacional Linux, estava descontente com o BitKeeper, o sistema para controle de versão no desenvolvimento do kernel Linux. O Git também é bastante utilizado em empresas em todo o mundo, inclusive no Brasil.

O uso dessa tecnologia favorece a rastreabilidade das modificações e a colaboração entre desenvolvedores.

5.5 GITHUB

Conforme afirma Ferreira (2014), o GitHub foi criado em 2008 como uma aplicação web para hospedagem de repositórios Git e colaboração entre desenvolvedores.

Diversos projetos de código aberto importantes são hospedados no GitHub, como jQuery, Node.js, Ruby on Rails, Jenkins, Spring, JUnit e muitos outros.

Atualmente, o GitHub se tornou uma das principais plataformas utilizadas por desenvolvedores e empresas para compartilhamento de código, controle de versões e colaboração em projetos de software.

5.6 BLADE (SISTEMA DE TEMPLATES)

Conforme Luciano (2016), grande parte dos frameworks web utiliza sistemas de templates para padronizar a interface visual das aplicações. Esse recurso otimiza a construção do projeto, facilitando o reuso de código e centralizando partes fixas da interface.

Com esse objetivo, o Laravel utiliza o Blade, um sistema de templates que se diferencia de outras soluções em PHP por não restringir o uso da linguagem PHP em suas páginas. Além disso, no Blade, cada view é compilada e armazenada em cache até sofrer alguma alteração, o que torna a aplicação mais eficiente e leve.

5.7 VISUAL STUDIO CODE

O Visual Studio Code é um editor de código-fonte desenvolvido pela Microsoft, amplamente utilizado no desenvolvimento de software. Ele oferece suporte a múltiplas linguagens de programação, integração com Git e extensões que aumentam sua funcionalidade Wikipédia (2026).

Devido à sua leveza e flexibilidade, tornou-se uma das ferramentas mais populares entre desenvolvedores web e back-end.

5.8 CONTROLE DE ACESSO BASEADO EM FUNÇÕES (RBAC)

O modelo de Controle de Acesso Baseado em Funções, conhecido pela sigla RBAC (*Role-Based Access Control*), é um método de autorização que gerencia o acesso de usuários a sistemas, aplicações e dados com base em funções predefinidas. Segundo IBM (s.d.), nesse modelo, o acesso não é atribuído diretamente ao indivíduo, mas ao papel que ele desempenha na organização.

Em um sistema que utiliza RBAC, um administrador atribui a cada usuário uma ou mais funções, em que cada função representa um conjunto específico de permissões ou privilégios. De acordo com IBM (s.d.), esse modelo oferece diversas vantagens:

- **Simplificação da gestão:** facilita o gerenciamento de acesso em organizações com muitos funcionários;

- **Segurança da informação:** auxilia na proteção contra ações internas indevidas e ameaças externas;
- **Princípio do menor privilégio:** garante que o usuário tenha acesso apenas aos recursos necessários ao exercício de sua função.

A aplicação do RBAC é fundamental para manter a integridade dos dados e a organização dos fluxos operacionais, permitindo que o sistema escale de forma segura à medida que novos perfis e permissões sejam necessários.

5.9 DOCKER

Segundo Vitalino e Castro (2018), o Docker surgiu em 2013 a partir da empresa dotCloud, fundada por Solomon Hykes. Inicialmente, a empresa atuava com soluções PaaS (*Platform as a Service*), permitindo suporte a diferentes linguagens de programação, diferentemente de outras plataformas da época.

Os autores destacam que o Docker nasceu como uma tecnologia open source baseada inicialmente em LXC (Linux Containers) e Union Filesystem, possibilitando a criação de containers leves e eficientes. Seu crescimento ocorreu rapidamente, tornando-se amplamente utilizado por empresas de grande porte, como Spotify, além de receber suporte de plataformas como AWS e Google.

De acordo com Vitalino e Castro (2018), o Docker trouxe maior facilidade na criação, distribuição e execução de aplicações em containers, garantindo mais portabilidade e consistência entre ambientes de desenvolvimento e produção. Além disso, por utilizar o mesmo kernel do sistema operacional hospedeiro, a tecnologia apresenta maior eficiência no consumo de recursos quando comparada às máquinas virtuais tradicionais.

Os autores também ressaltam que o Docker possui melhor desempenho em sistemas Linux, embora atualmente também possa ser executado em ambientes Windows e macOS. Dessa forma, o Docker tornou-se uma das principais ferramentas utilizadas em processos modernos de desenvolvimento e implantação de software.

5.10 REDIS

O Redis (*Remote Dictionary Server*) é um banco de dados NoSQL em memória amplamente utilizado como sistema de cache, armazenamento de sessões, filas de mensagens e banco de dados chave-valor. Diferentemente de outras soluções de cache, o Redis oferece suporte a diversas estruturas de dados, como listas, conjuntos, hashes e conjuntos ordenados, além de permitir persistência opcional dos dados em disco.

Segundo Redis Ltd. (2025), o armazenamento dos dados diretamente na memória RAM reduz o tempo de acesso em comparação com sistemas que dependem de operações em disco, tornando o Redis adequado a aplicações que exigem alto desempenho.

Além disso, o Redis oferece mecanismos de expiração automática de chaves (*Time To Live – TTL*), permitindo que os dados armazenados sejam removidos automaticamente após determinado período, evitando inconsistências e desperdício de memória.

5.10.1 Cache em Aplicações Web

O cache é uma técnica utilizada para reduzir o tempo de resposta das aplicações, armazenando temporariamente informações frequentemente acessadas em uma camada de memória de alta velocidade. Dessa forma, evita-se a execução repetitiva de consultas ao banco de dados e outros processos computacionalmente custosos, contribuindo para a melhoria do desempenho e da escalabilidade dos sistemas.

Em aplicações web modernas, o uso de mecanismos de cache torna-se essencial principalmente em cenários com elevado número de acessos simultâneos. A utilização dessa estratégia reduz a carga sobre o banco de dados, diminui a latência das requisições e melhora a experiência do usuário.

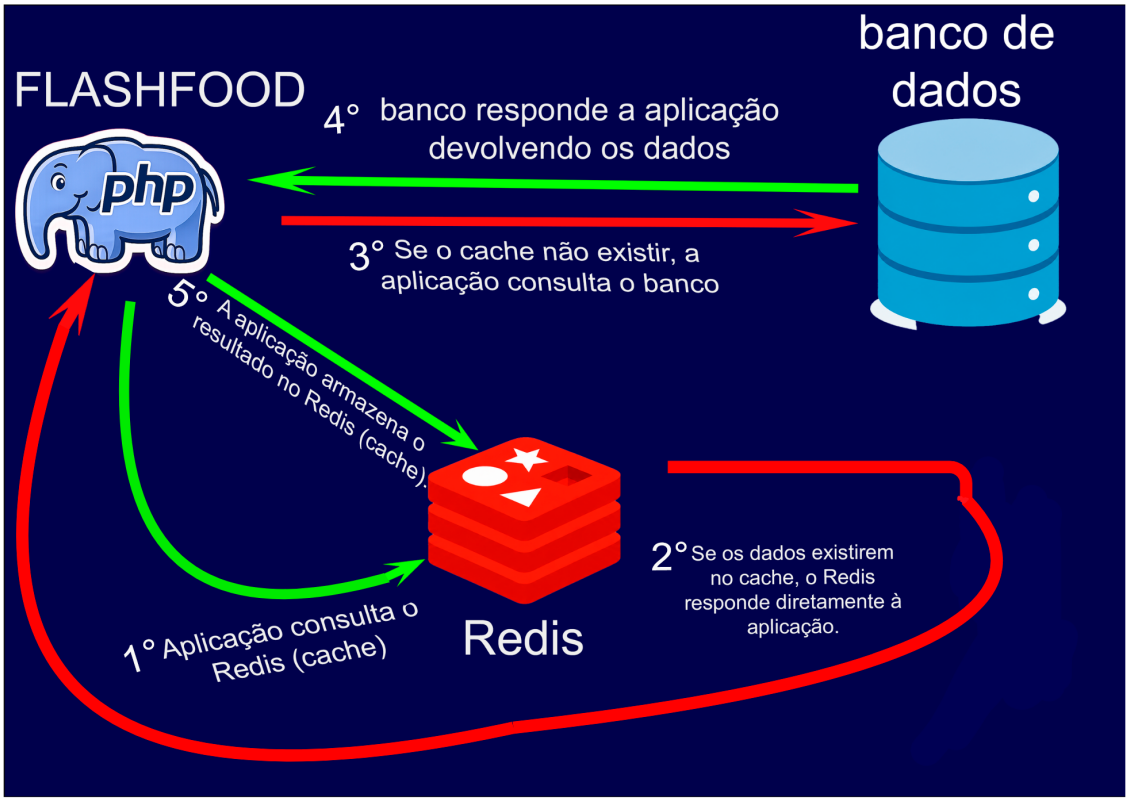
5.10.2 Redis como Camada de Cache

O Redis é frequentemente utilizado como uma camada intermediária entre a aplicação e o banco de dados. Quando uma informação é solicitada, a aplicação verifica inicialmente se ela está armazenada no cache. Caso esteja disponível, o dado é

retornado imediatamente. Caso contrário, a informação é recuperada do banco de dados, armazenada no Redis e posteriormente enviada ao usuário.

O fluxo de consulta ao cache e ao banco de dados adotado no sistema Flashfood é apresentado na Figura 1.

Figura 1 – Fluxo de consulta ao Redis no sistema Flashfood



Fonte: Elaborado pelo autor (2026)..

Esse processo reduz significativamente a quantidade de consultas ao banco de dados, especialmente em informações acessadas repetidamente, como listas de produtos, categorias, configurações do sistema e dados estatísticos.

De acordo com Almeida *et al.* (2023), o Redis apresenta desempenho superior em operações de leitura quando comparado a bancos relacionais tradicionais, tornando-se uma alternativa para aplicações com alta demanda de acesso.

5.10.3 Escalabilidade Horizontal

A escalabilidade horizontal consiste na ampliação da capacidade de processamento de uma aplicação por meio da adição de novos servidores, em vez do aumento da capacidade de um único equipamento. Nesse cenário, soluções de cache como o Redis desempenham papel fundamental, reduzindo o número de acessos simultâneos ao banco de dados e distribuindo de forma mais eficiente a carga entre os servidores.

Segundo Filho (2025), arquiteturas que utilizam cache podem responder mais rapidamente às requisições dos usuários, proporcionando maior disponibilidade e desempenho.

5.10.4 Aplicação do Redis no Sistema Desenvolvido

Neste trabalho, o Redis foi utilizado como mecanismo de cache da aplicação desenvolvida em Laravel. Informações frequentemente consultadas, como listas de funcionários, categorias e outros registros de baixa frequência de alteração, foram armazenadas temporariamente no Redis utilizando o mecanismo `Cache::remember()` disponibilizado pelo framework.

Quando ocorre alteração nos dados, como inclusão, edição ou exclusão de registros, o cache correspondente é invalidado por meio do método `Cache::forget()`, garantindo que as informações apresentadas aos usuários permaneçam consistentes.

Essa estratégia reduz a quantidade de consultas ao banco de dados MySQL, melhora o tempo de resposta da aplicação e aumenta sua capacidade de atender múltiplos usuários simultaneamente.

6 REQUISITOS DE SOFTWARE

Nesta seção são apresentados os requisitos de software definidos para o sistema, considerando as funcionalidades que devem ser disponibilizadas aos usuários e as características necessárias para seu funcionamento. Os requisitos são divididos em funcionais e não funcionais.

6.1 REQUISITOS FUNCIONAIS

Os requisitos funcionais de um sistema descrevem o que ele deve fazer, ou seja, os serviços e funcionalidades que o sistema deve fornecer aos usuários. Esses requisitos podem variar de descrições mais gerais até especificações detalhadas, incluindo entradas, saídas e comportamentos esperados do sistema.

Quando expressos como requisitos de usuário, são apresentados de forma mais abstrata para facilitar o entendimento. Já em nível de sistema, tornam-se mais detalhados e técnicos. A imprecisão na definição desses requisitos pode gerar interpretações diferentes pelos desenvolvedores, causando problemas, retrabalho e aumento de custos.

Em sistemas complexos, é difícil garantir que os requisitos funcionais sejam totalmente completos e consistentes, devido à grande quantidade de stakeholders e possíveis conflitos entre necessidades. Por isso, falhas ou omissões podem aparecer durante o desenvolvimento ou após a entrega do sistema (SOMMERVILLE, 2011).

Com base nas necessidades identificadas e nos objetivos definidos para o sistema, foram estabelecidos os requisitos funcionais apresentados na Tabela 1.

6.2 REQUISITOS NÃO FUNCIONAIS

Os requisitos não funcionais não estão diretamente relacionados às funções específicas do sistema, mas às características de qualidade e restrições globais, como desempenho, confiabilidade, segurança, tempo de resposta e disponibilidade. Esses requisitos podem também envolver limitações de implementação, como tipos de dis-

Tabela 1 – Requisitos funcionais do sistema

ID	Descrição
RF01	O sistema deve permitir o cadastro de usuários, incluindo funcionários e clientes.
RF02	O sistema deve permitir que os usuários realizem autenticação e encerramento de sessão.
RF03	O sistema deve controlar o acesso às funcionalidades de acordo com o perfil atribuído ao usuário.
RF04	O sistema deve permitir que usuários gerenciem pedidos.
RF05	O sistema deve exibir os produtos disponíveis na área do cliente.
RF06	O sistema deve permitir gerenciar o carrinho de compras.
RF07	O sistema deve permitir a confirmação e o registro de pedidos.
RF08	O sistema deve permitir que funcionários autorizados consultem os pedidos registrados.
RF09	O sistema deve permitir a localização e o acompanhamento dos pedidos em andamento.
RF10	O sistema deve permitir a atualização do status do pedido para pendente, em preparo ou finalizado.
RF11	O sistema deve armazenar e recuperar os dados de usuários, produtos e pedidos.

Fonte: Fonte: Elaborado pelo autor (2026)..

positivos de entrada e saída e formas de representação de dados em interfaces com outros sistemas.

Diferentemente dos requisitos funcionais, os requisitos não funcionais são importantes para o funcionamento adequado do sistema como um todo. Falhas relacionadas à segurança, à disponibilidade ou ao desempenho podem comprometer o atendimento, a integridade dos dados e a continuidade das operações da lanchonete.

Além disso, esses requisitos são mais difíceis de serem associados a componentes específicos do sistema, pois sua implementação geralmente está distribuída em várias partes do software (SOMMERVILLE, 2011).

Os requisitos não funcionais definidos para o sistema são apresentados na Tabela 2.

Tabela 2 – Requisitos não funcionais do sistema

ID	Descrição
RNF01	O sistema deve suportar múltiplos usuários simultâneos.
RNF02	As senhas devem ser armazenadas exclusivamente na forma de resumo criptográfico seguro (*hash*), conforme os mecanismos disponibilizados pelo Laravel.
RNF03	O sistema deve funcionar 24 horas por dia, 7 dias por semana.
RNF04	O sistema deve funcionar em navegadores como Chrome, Firefox e Edge.
RNF05	O código-fonte deve seguir a organização do padrão MVC, de modo a separar apresentação, regras de negócio e acesso aos dados.
RNF06	O sistema deve proteger as rotas e operações restritas por meio de autenticação e autorização baseada no perfil do usuário.
RNF07	O sistema deve registrar logs de autenticação e das operações de inclusão, alteração e exclusão de registros.

Fonte: Fonte: Elaborado pelo autor (2026)..

7 MODELAGEM DE SOFTWARE

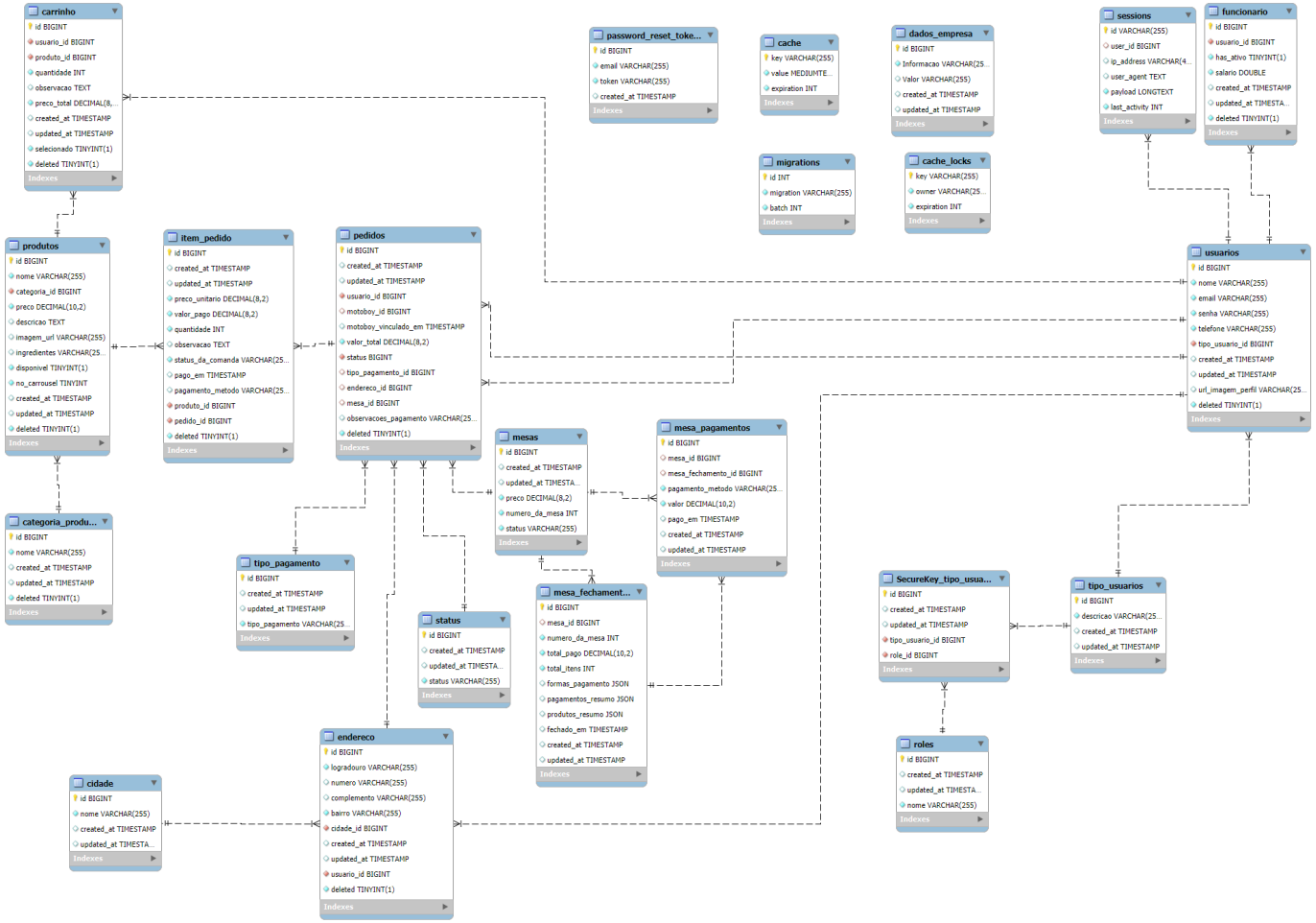


Figura 2 – Modelo físico do banco de dados do sistema Flashfood

Fonte: Elaborado pelo autor (2026)..

8 CONCLUSÃO

Este trabalho apresentou a proposta e o desenvolvimento de uma aplicação web para apoiar a gestão operacional de uma lanchonete. A solução foi concebida para centralizar informações de usuários, produtos e pedidos, reduzindo a dependência de registros manuais e favorecendo a organização das rotinas do estabelecimento.

A adoção do Laravel e do padrão MVC contribuiu para separar as responsabilidades da aplicação. O MySQL foi utilizado para a persistência dos dados, enquanto o Redis foi incorporado como camada de cache para informações consultadas com frequência. O controle de acesso baseado em funções permitiu relacionar as permissões às atribuições dos diferentes usuários. Docker, Git e GitHub complementaram o ambiente de desenvolvimento ao apoiar, respectivamente, a padronização da execução e o controle de versões.

Os requisitos levantados abrangem o cadastro e a autenticação de usuários, o gerenciamento e a exibição de produtos, o carrinho de compras e o acompanhamento dos pedidos. Esses recursos atendem às necessidades centrais identificadas no contexto analisado. Entretanto, a comprovação dos ganhos de desempenho, usabilidade e eficiência operacional depende da execução e da documentação de testes em ambiente representativo da lanchonete.

Como trabalhos futuros, recomenda-se avaliar o sistema com usuários reais, mensurar o tempo das operações antes e depois de sua implantação e ampliar as funcionalidades de estoque, comandas, pagamentos e relatórios gerenciais. Também podem ser investigadas integrações com meios de pagamento e serviços de entrega, desde que compatíveis com as necessidades do estabelecimento.

REFERÊNCIAS

ALMEIDA, D. *et al.* Performance comparison of redis, memcached, mysql, and postgresql: A study on key-value and relational databases. In: **Second International Conference on Smart Technologies for Smart Nation**. [S.l.: s.n.], 2023. p. 902–907.

DALL'OGLIO, P. **PHP: programando com orientação a objetos**. São Paulo: Novatec Editora, 2015. 552 p.

FERREIRA, R. **Controlando versões com Git e GitHub**. [S.l.]: Casa do Código, 2014. 199 p.

FILHO, N. S. Comparative analysis between monolithic architecture and microservices in .net applications. **Leaders Tec**, v. 2, n. 13, 2025.

IBM. **O que é controle de acesso baseado em função (RBAC)?** s.d. Acesso em: 30 abr. 2026. Disponível em: <https://www.ibm.com/>.

LUCIANO. **Blade Engine: Utilizando templates no Laravel**. 2016. Acesso em: 29 abr. 2026. Disponível em: <https://www.devmedia.com.br/blade-engine-utilizando-templates-no-laravel/36749>.

MILANI, A. **MySQL: guia do programador**. São Paulo: Novatec Editora, 2007. 400 p.

N.RICARDO. **O que é Laravel: principais recursos, onde usar e vantagens do framework PHP**. 2025. Acesso em: 29 abr. 2026. Disponível em: <https://www.hostinger.com/br/tutoriais/o-que-e-laravel>.

Redis Ltd. **Redis Documentation**. 2025. Disponível em: <https://redis.io/documentation>.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011. Revisão técnica de Kechi Hirama.

VITALINO, J. F. N.; CASTRO, M. A. N. **Descomplicando o Docker**. 2. ed. Rio de Janeiro: Brasport, 2018. ISBN 978-85-7452-902-8.

Wikipédia. **Visual Studio Code**. 2026. Acesso em: 29 abr. 2026. Disponível em: <https://pt.wikipedia.org/wiki/Visual_Studio_Code>.